

LE LISTE

INTRODUZIONE

COSA FA IL CODICE?

COSA FA IL CODICE?

COSA FA IL CODICE?

DOMANDE

TERMINI CHIAVE DELLA LEZIONE

DA RICORDARE IN PYTHON

ESERCIZI

INTRODUZIONE

In questa lezione vedremo come gestire un insieme di dati mediante una struttura dati chiamata **lista** che ci permette, con un unico nome, di indicare una sequenza di dati. I dati all'interno della lista vengono identificati con un **indice** che rappresenta la posizione in cui si trovano. L'elemento in prima posizione ha indice 0.

Per indicare che i dati sono contenuti in una lista si utilizzano le parentesi quadre.

Esempio:

```
anni = [2020, 2021, 2022, 2023]
```

indice:	0	1	2	3
elemento:	2020	2021	2022	2023
indice negativo:	-4	-3	-2	-1

L'elemento 2020 si trova nella posizione di indice 0

→ per richiamarlo scriviamo `anni[0]`

Analogamente il 2023 si trova nella posizione di indice 3

→ per richiamarlo scriviamo `anni[3]`

E' possibile usare anche gli indici negativi indicizzando la lista a partire dal fondo, per esempio possiamo indicare il 2023 come elemento in posizione -1

→ per richiamarlo scriviamo `anni[-1]`

In questa lezione incontrerete alcune funzioni per gestire le liste. Al solito, per immaginare il loro scopo, potete rifarvi alla traduzione della parola inglese.

Per esempio:

- `append()` significa *accodare*
- `len()` sta per *length* che significa *lunghezza*

COSA FA IL CODICE?

Lavoro in coppia

Osservate il seguente codice:

```
studenti = ["Arianna", "Bruno", "Dario", "Elena", "Mario", "Teresa"]
for nome in studenti:
    print(nome)
print(studenti)
print(studenti[0])
print(studenti[1])
print(studenti[1:4])
```

Secondo voi, cosa viene visualizzato dall'istruzione `print(nome)` ?

Secondo voi, cosa viene visualizzato dall'istruzione `print(studenti)` ?

Secondo voi, cosa viene visualizzato dalle istruzioni `print(studenti[0])` e `print(studenti[1])` ?

Secondo voi, cosa viene visualizzato dall'istruzione `print(studenti[1:4])` ?

Copiate e incollate il codice in Thonny (salvate con nome *135_Liste.py*) e controllate cosa fa. Fa quello che vi aspettavate? Provate a dare una spiegazione in caso contrario.

Osservate ora le seguenti istruzioni:

```
studenti[0]="Giovanni"  
studenti.append("Giorgia")  
print(studenti)
```

Cosa verrà visualizzato dall'istruzione `print(studenti)`?

Ora copiate e incollate il codice in Thonny (in fondo al file `135_Liste.py`) e controllate cosa fa. Fa quello che vi aspettavate? Provate a dare una spiegazione in caso contrario.

Osservate ora le seguenti istruzioni:

```
studenti[1]=2  
print(studenti)
```

Cosa verrà visualizzato dall'istruzione `print(studenti)`?

Ora copiate e incollate il codice in Thonny (in fondo al file `135_Liste.py`) e controllate cosa fa. Fa quello che vi aspettavate? Provate a dare una spiegazione in caso contrario.

COSA FA IL CODICE?

Lavoro in coppia

Osservate il seguente codice:

```
studenti = ["Arianna", "Bruno", "Dario", "Elena", "Mario", "Teresa"]
for i in range(len(studenti)):
    print(i, studenti[i])
```

Secondo voi, cosa fa il codice?
Copiate e incollate il codice in Thonny (salvate con nome <i>136_Len.py</i>) e controllate cosa fa. Fa quello che vi aspettavate? Provate a dare una spiegazione in caso contrario.

COSA FA IL CODICE?

Lavoro in coppia

Osservate il seguente codice:

```
studenti = ["Arianna", "Bruno", "Dario", "Elena", "Mario", "Teresa"]
studente_cercato=input("Scrivi il nome dello studente: ")
if studente_cercato in studenti:
    print(studente_cercato, "è uno studente di questa scuola")
else:
    print(studente_cercato, "non è un nostro studente")
```

Secondo voi, cosa fa il codice?
Copiate e incollate il codice in Thonny (salvate con nome <i>137_In.py</i>) e controllate cosa fa. Fa quello che vi aspettavate? Provate a dare una spiegazione in caso contrario.
Provate a sostituire <code>in</code> con l'istruzione <code>not in</code> . Cosa succede di diverso?

DOMANDE

Lavoro in coppia

Rispondete alle seguenti domande:

1. Le liste possono cambiare il loro contenuto?
2. Le liste possono contenere elementi di tipo diverso (es. stringhe insieme a numeri)?
3. Che differenza c'è tra i due cicli che abbiamo visto per lo scorrimento delle liste:

```
for x in studenti e for x in range(len(studenti))?
```

TERMINI CHIAVE DELLA LEZIONE

TERMINE	SIGNIFICATO	ESEMPIO IN PYTHON
<code>lista</code>	sequenza di dati che sono salvati in un unico contenitore	<code>lista = [3,2,5]</code>
<code>lista.append(elemento)</code>	Aggiunge elemento alla fine della lista	<code>lista = [3,2,5]</code> <code>lista.append(8)</code> <code>print(lista)</code> → <code>[3,2,5,8]</code>
<code>lista[i]</code>	Identifica l'elemento nella posizione <code>i</code> della lista	<code>lista = [3,2,5,8]</code> <code>print(lista[2])</code> → <code>5</code>
<code>lista[i:j]</code>	Accede a una parte di lista che va dall'elemento di indice <code>i</code> all'elemento di indice <code>j</code> (escluso). Questa operazione si chiama slicing .	<code>lista=[3,2,5,8,10]</code> <code>print(lista[1:3])</code> → <code>[2,5]</code>
<code>lista[-i]</code>	Identifica l'elemento in posizione <code>i</code> partendo dalla fine	<code>lista=[3,2,5,8,10]</code> <code>print(lista[-2])</code> → <code>8</code>
<code>lista.insert(n,elemento)</code>	Inserisce elemento nella posizione <code>n</code> della lista	<code>lista = [3,2,7,8]</code> <code>lista.insert(1,10)</code> <code>print(lista)</code> → <code>[3,10,2,7,8]</code>
<code>lista.remove(elemento)</code>	Rimuove elemento da lista. Se c'è più di un elemento uguale a quello indicato, elimina il primo che trova (quello nella posizione più vicina allo 0).	<code>lista = [3,10,2,7]</code> <code>lista.remove(2)</code> <code>print(lista)</code> → <code>[3,10,7]</code>

<code>del(lista[i])</code>	Rimuove l'elemento di posizione <code>i</code> nella lista	<code>lista = [3,10,7,8]</code> <code>del(lista[3])</code> <code>print(lista)</code> → <code>[3,10,7]</code>
<code>len(lista)</code>	Restituisce il numero di elementi della lista	<code>lista = [3,2,5]</code> <code>print(len(lista))</code> → <code>3</code>

DA RICORDARE IN PYTHON

Le liste sono strutture di dati che consentono di **assegnare un gruppo di valori a un identificatore**. Questo evita l'uso di più variabili per memorizzare un gruppo di valori.

Le liste possono **crescere e ridursi** di dimensioni e memorizzare gruppi di valori di **tipi di dati misti**.

Gli elementi delle liste possono essere modificati: le **liste** sono infatti strutture dati **mutabili**.

Una lista si rappresenta elencando gli elementi che la compongono in sequenza, separati da una virgola. La sequenza è racchiusa in una coppia di parentesi quadre.

Esempi:

```
dati = [1, 2, "mele", "fragole"]
```

```
frutta = ["mele", "pere", "arance"]
```

```
anni = [2020, 2021, 2022, 2023]
```

Come usare le strutture che conosciamo per manipolare le liste:

<code>for elemento in lista:</code>	scorre gli elementi della lista, si usa quando il valore dell'indice dell'elemento non è necessario
<code>for i in range(len(studenti)):</code>	scorre gli elementi della lista, si usa quando serve conoscere l'indice (posizione) dell'elemento della lista
<code>if elemento in lista:</code>	verifica se un elemento è nella lista
<code>if elemento not in lista:</code>	verifica se un elemento NON è nella lista

ESERCIZI

Lavoro in autonomia

1. [138_ManipolaListe.py] Crea una lista (chiamala *studenti*) e inserisci i nomi Arianna, Bruno, Dario, Elena, Mario, Teresa. Poi scrivi le istruzioni per eseguire le seguenti operazioni:
 - aggiungi in fondo alla lista il nome tuo e del tuo vicino di banco
 - inserisci in posizione di indice 3 il nome di un altro tuo compagno di classe
 - rimuovi lo studente in seconda posizione
 - cancella la studentessa "Teresa"
 - visualizza posizione e nome di ogni studente della lista
2. [139_Studenti.py] Crea una lista (chiamala *studenti*) che sia inizialmente vuota e una seconda lista con i nomi di 5 tuoi compagni di classe (chiamala *studenti_classe*). Chiedi poi all'utente di inserire 5 nomi (con un ciclo for). Verifica se ogni nome inserito è presente in *studenti_classe*. In caso affermativo aggiungilo in fondo alla lista *studenti*, altrimenti stampa "questo studente non va inserito nella classe". Prima di inserire lo studente nella lista *studenti*, verifica anche che non sia già stato inserito precedentemente.
3. [140_ListaCas.py] Crea una lista di 10 numeri generati casualmente (scegli tu gli estremi dell'intervallo in cui generi i numeri e il nome della lista). Visualizza gli elementi dalla prima all'ultima posizione e poi dall'ultima posizione alla prima.
4. [141_ListaQuadrati.py] L'utente inserisce un numero N, il programma genera N numeri casuali (tra 1 e 20) e li salva in una lista. Poi, il programma visualizza tutti i numeri generati e poi tutti i numeri elevati alla seconda.
5. [142_ListaRaddoppia.py] Crea una lista di 10 numeri interi casuali in un intervallo a tua scelta e raddoppia i valori di quelli dispari.
6. [143_SommaLista.py] L'utente inserisce un numero N, il programma genera N numeri casuali (tra 1 e 20) e li salva in una lista. Poi, il programma calcola la somma dei numeri contenuti nella lista. *Suggerimento: puoi partire dall'esercizio 114_SommaN.py e inserire al suo interno la gestione delle liste e la generazione casuale.*

7. [144_PosNeg.py] L'utente inserisce un numero N, il programma genera N numeri casuali (tra -20 e 20) e li salva in una lista. Poi, il programma calcola separatamente la somma dei numeri positivi e quella dei numeri negativi della lista.
8. [145_PariDispari.py] Chiedi in input all'utente quanti numeri (N) vuole nella lista, poi genera casualmente N numeri casuali e inseriscili in una lista. Stabilisci quanti elementi sono pari e quanti dispari nella lista. Visualizza la lista generata e il numero di pari e di dispari. *Suggerimento puoi partire dall'esercizio 126_PariDispari.py e inserire il codice per gestire le liste e la generazione casuale.*
9. [146_istogramma.py] Crea una lista di numeri e stampa un istogramma basato su questi numeri, usando asterischi per disegnarlo. (Scegli tu come creare la lista). Ad esempio, data la lista [3,7,9] il programma deve produrre questo grafico:

10. [147_MaxLista.py] Generare casualmente una lista di 20 numeri tra 1 e 100. Determinare il numero massimo della lista. Visualizza la lista generata e il valore massimo trovato. *Suggerimento: puoi partire dall'esercizio 124_MassimoNumero.py e inserire al suo interno la gestione delle liste e la generazione casuale.*
11. [148_Prodotto.py] Crea una lista di 10 numeri interi random tra 1 e 10 e calcola il prodotto dei numeri.
12. [149_SopraMedia.py] Crea una lista di 10 numeri interi random tra 1 e 10 e conta i componenti che hanno un valore superiore alla media (calcolata su tutta la lista).